

راهنمای کامل ساخت اپلیکیشن «تایمر قفل گوشی» با Kotlin

۱۱ شهریور ۱۴۰۵

فهرست مطالب

- ۱ خلاصه پروژه و معماری کلی
- ۲ پیکربندی پروژه و فایل AndroidManifest
- ۳ کلاس AdminReceiver و سیاست‌های دستگاه
- ۴ سرویس تایمر پس‌زمینه و قفل خودکار
- ۵ رابط کاربری و شروع تایمر
- ۶ تست، سوالات پرتکرار و نکات انتشار

۱ خلاصه پروژه و معماری کلی

این سند یک برنامه‌نویس مبتدی تا متوسط را قدم‌به‌قدم در ساخت یک اپلیکیشن اندرویدی با نام «تایمر قفل» همراهی می‌کند: کاربر یک مدت‌زمان (مثلاً ۶۰ دقیقه) تعیین می‌کند و پس از پایان آن، گوشی به‌صورت خودکار به صفحه اصلی برمی‌گردد و صفحه قفل می‌شود. تمام کدهای لازم در همین سند ارائه شده و آماده کپی‌کردن در پروژه است.

معماری پروژه روی سه ستون استوار است:

۱. DevicePolicyManager و DeviceAdminReceiver – تنها راه استاندارد و بدون نیاز به روت برای قفل کردن صفحه در اندروید. متد lockNow() با دسترسی ادمین صفحه را فوراً خاموش و قفل می‌کند.
۲. Foreground Service – اندروید برای صرفه‌جویی در باتری، فرآیندهای پس‌زمینه را متوقف می‌کند. برای اینکه تایمر حتی وقتی کاربر در برنامه‌های دیگر است دقیق شمرده شود، از یک سرویس foreground با نوتیفیکیشن دائمی استفاده می‌کنیم.
۳. Jetpack Compose – رابط کاربری ساده برای انتخاب مدت‌زمان و شروع/لغو تایمر.

نکته مهم امنیتی: اندروید به هیچ اپلیکیشنی اجازه نمی‌دهد فرآیندهای دیگر را مستقیماً ببندد (Kill کند). بنابراین استراتژی درست این است که در لحظه پایان تایمر، کاربر با یک Intent به صفحه Home برگردانده شود و بلافاصله صفحه قفل گردد؛ برنامه‌های باز در پس‌زمینه می‌مانند ولی کاربر دیگر به آن‌ها دسترسی ندارد.

۲ پیکربندی پروژه و فایل AndroidManifest

ابتدا Android Studio (نسخه Koala یا جدیدتر) را نصب کنید و پروژه جدیدی با تنظیمات زیر بسازید:

- قالب: Empty Activity (Compose)
- زبان: Kotlin
- نام اپلیکیشن: LockTimer
- پکیج: com.example.locktimer
- Minimum SDK: API 26 (اندروید ۸ به بالا)

سپس فایل AndroidManifest.xml را طوری ویرایش کنید که محتوای داخل تگ <application> چنین شود:

```
<receiver
    android:name=".AdminReceiver"
    android:permission="android.permission.BIND_DEVICE_ADMIN"
    android:exported="true">
    <meta-data
        android:name="android.app.device_admin"
        android:resource="@xml/device_admin_policy" />
    <intent-filter>
        <action android:name="android.app.action.DEVICE_ADMIN_ENABLED" />
    </intent-filter>
</receiver>

<service
    android:name=".LockTimerService"
    android:exported="false"
    android:foregroundServiceType="specialUse">
    <property
        android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE"
        android:value="countdown timer for screen lock" />
</service>
```

داخل <manifest> (خارج از application) این مجوزها را اضافه کنید:

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

مجوز POST_NOTIFICATIONS از اندروید ۱۳ به بعد باید در زمان اجرا از کاربر گرفته شود؛ در بخش رابط کاربری این کار انجام شده است.

۳ کلاس AdminReceiver و سیاست‌های دستگاه

دو فایل زیر را در پروژه ایجاد کنید:

(۱) فایل res/xml/device_admin_policy.xml (پوشه xml را در res بسازید):

```
<?xml version="1.0" encoding="utf-8"?>
<device-admin>
    <uses-policies>
        <force-lock />
    </uses-policies>
</device-admin>
```

با استفاده از سیاست force-lock فقط اجازه قفل کردن صفحه را درخواست می‌کنیم و هیچ دسترسی خطرناک دیگری (مثل پاک کردن داده‌ها) مطرح نیست؛ این امر اعتماد کاربر هنگام فعال‌سازی ادمین را بالا می‌برد.

۲) کلاس AdminReceiver.kt

```
package com.example.locktimer

import android.app.admin.DeviceAdminReceiver

class AdminReceiver : DeviceAdminReceiver() {
    //
    //
}
```

در نخستین اجرای برنامه، باید کاربر را با Intent مخصوص ACTION_ADD_DEVICE_ADMIN به صفحه فعال‌سازی ادمین بفرستیم (کد کامل آن در بخش رابط کاربری آمده است). این فعال‌سازی یک‌بار برای همیشه است و تا زمانی که کاربر دستی لغو نکند باقی می‌ماند.

۴ سرویس تایمر پس‌زمینه و قفل خودکار

این هسته اصلی برنامه است. فایل LockTimerService.kt را بسازید:

```
package com.example.locktimer

import android.app.*
import android.content.Context
import android.content.Intent
import android.os.*
import androidx.core.app.NotificationCompat

class LockTimerService : Service() {

    private var handler: Handler? = null
    private var countdownRunnable: Runnable? = null

    companion object {
        const val EXTRA_MINUTES = "extra_minutes"
        const val CHANNEL_ID = "lock_timer_channel"
        const val NOTIFICATION_ID = 1
        const val ACTION_CANCEL = "com.example.locktimer.CANCEL"
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        if (intent?.action == ACTION_CANCEL) {
            stopSelf()
            return START_NOT_STICKY
        }

        val minutes = intent?.getIntExtra(EXTRA_MINUTES, 60) ?: 60
        startForeground(NOTIFICATION_ID, buildNotification(minutes))
    }
}
```

```

handler = Handler(Looper.getMainLooper())
var remainingSeconds = minutes * 60L

countdownRunnable = object : Runnable {
    override fun run() {
        remainingSeconds--
        if (remainingSeconds <= 0) {
            lockScreenNow()
            stopSelf()
            return
        }
        if (remainingSeconds % 60 == 0L) {
            updateNotification((remainingSeconds / 60).toInt())
        }
        handler?.postDelayed(this, 1000)
    }
}
handler?.postDelayed(countdownRunnable!!, 1000)
return START_STICKY
}

private fun lockScreenNow() {
    //
    val homeIntent = Intent(Intent.ACTION_MAIN).apply {
        addCategory(Intent.CATEGORY_HOME)
        flags = Intent.FLAG_ACTIVITY_NEW_TASK
    }
    startActivity(homeIntent)

    //
    val dpm = getSystemService(Context.DEVICE_POLICY_SERVICE) as DevicePolicyManager
    val compName = ComponentName(this, AdminReceiver::class.java)
    if (dpm.isAdminActive(compName)) {
        dpm.lockNow()
    }
}

private fun buildNotification(minutes: Int): Notification {
    val ch = NotificationChannel(
        CHANNEL_ID, " ",
        NotificationManager.IMPORTANCE_LOW
    )
    getSystemService(NotificationManager::class.java).createNotificationChannel(ch)

    val cancelIntent = PendingIntent.getService(
        this, 0,
        Intent(this, LockTimerService::class.java).setAction(ACTION_CANCEL),
        PendingIntent.FLAG_IMMUTABLE
    )

```

```

        return NotificationCompat.Builder(this, CHANNEL_ID)
            .setContentTitle("          ("
            .setContentText("          $minutes          -          ("
            .setSmallIcon(android.R.drawable.ic_lock_lock)
            .setOngoing(true)
            .addAction(0, " ", cancelIntent)
            .build()
    }

    private fun updateNotification(minutes: Int) {
        val nm = getSystemService(NotificationManager::class.java)
        nm.notify(NOTIFICATION_ID, buildNotification(minutes))
    }

    override fun onBind(intent: Intent?): IBinder? = null

    override fun onDestroy() {
        handler?.removeCallbacksAndMessages(null)
        super.onDestroy()
    }
}

```

چند نکته فنی: `Handler.postDelayed` در یک سرویس `foreground` برای شمارش دقیقه‌ای کافی است؛ اگر به دقت ثانیه‌ای در شرایط خواب عمیق دستگاه نیاز دارید، به جای آن از `AlarmManager.setExactAndAllowWhileIdle` به عنوان پشتیبان استفاده کنید. اکشن «لغو تایمر» روی نوتیفیکیشن باعث می‌شود کاربر هر زمان بخواهد قفل خودکار را قفل خود کند.

۵ رابط کاربری و شروع تایمر

در رابط کاربری با `Compose`، کاربر دقیقه‌ها را وارد می‌کند، مجوز ادمین و نوتیفیکیشن در صورت نیاز گرفته می‌شود و سرویس اجرا می‌شود. فایل `MainActivity.kt`:

```

package com.example.locktimer

import android.app.admin.DevicePolicyManager
import android.content.ComponentName
import android.content.Context
import android.content.Intent
import android.os.Build
import android.os.Bundle
import android.Manifest
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier

```

```
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {

    private val notifPermission =
        registerForActivityResult(ActivityResultContracts.RequestPermission()) { }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // +
        if (Build.VERSION.SDK_INT >= 33) {
            notifPermission.launch(Manifest.permission.POST_NOTIFICATIONS)
        }

        setContent {
            MaterialTheme {
                var minutes by remember { mutableStateOf("60") }

                Column(
                    modifier = Modifier.fillMaxSize().padding(24.dp),
                    horizontalAlignment = Alignment.CenterHorizontally,
                    verticalArrangement = Arrangement.Center
                ) {
                    Text("      ", fontSize = 24.sp)
                    Spacer(Modifier.height(24.dp))

                    OutlinedTextField(
                        value = minutes,
                        onChange = { minutes = it.filter { c -> c.isDigit() } },
                        label = { Text("      (" ) }
                    )
                    Spacer(Modifier.height(16.dp))

                    Button(onClick = {
                        val min = minutes.toIntOrNull()?.coerceIn(1, 720) ?: 60
                        if (requestAdmin()) startTimer(min)
                        else Toast.makeText(
                            this, "      ", Toast.LENGTH_LONG
                        ).show()
                    }) {
                        Text("      ("
                    }
                }
            }
        }

        private fun isAdminActive(): Boolean {
            val dpm = getSystemService(Context.DEVICE_POLICY_SERVICE) as DevicePolicyManager
```

```

        return dpm.isAdminActive(ComponentName(this, AdminReceiver::class.java))
    }

    private fun requestAdmin(): Boolean {
        if (isAdminActive()) return true
        val intent = Intent(DevicePolicyManager.ACTION_ADD_DEVICE_ADMIN).apply {
            putExtra(
                DevicePolicyManager.EXTRA_DEVICE_ADMIN,
                ComponentName(this@MainActivity, AdminReceiver::class.java)
            )
            putExtra(
                DevicePolicyManager.EXTRA_ADD_EXPLANATION,
                " "
            )
        }
        startActivity(intent)
        return false
    }

    private fun startTimer(minutes: Int) {
        val intent = Intent(this, LockTimerService::class.java)
            .putExtra(LockTimerService.EXTRA_MINUTES, minutes)
        if (Build.VERSION.SDK_INT >= 26) startForegroundService(intent)
        else startService(intent)
        Toast.makeText(this, " $minutes " , Toast.LENGTH_SHORT).show()
    }
}

```

جریان کاربری این است: در نخستین استفاده، دکمه «شروع تایمر» کاربر را به صفحه سیستم فعال سازی ادمین می برد؛ پس از فعال سازی، فشردن مجدد دکمه سرویس را با مدت زمان انتخابی اجرا می کند و نوتیفیکیشن شمارش معکوس در نوار وضعیت ظاهر می شود.

۶ تست، سوالات پرتکرار و نکات انتشار

برای تست، از **گوشی واقعی** استفاده کنید که با USB متصل شده باشد؛ شبیه سازها گاهی رفتار `lockNow()` و سرویس های foreground را دقیقاً شبیه دستگاه واقعی بازسازی نمی کنند. برای تست سریع، مدت زمان را روی ۱ یا ۲ دقیقه بگذارید.

نکات مهم و پرتکرار:

- **کرش هنگام شروع سرویس:** اگر خطای `ForegroundServiceStartNotAllowedException` گرفتید، مطمئن شوید مجوز `FOREGROUND_SERVICE_SPECIAL_USE` و `foregroundServiceType` در مانیفست ثبت شده اند.
- **قفل انجام نمی شود:** به بخش تنظیمات امنیت مدیران دستگاه بروید و فعال بودن اپلیکیشن را بررسی کنید.
- **کشدن تایمر توسط باتری:** در برخی گوشی ها (سامسونگ، شیائومی) باید در تنظیمات باتری، بهره وری اپ را روی «بدون محدودیت» بگذارید.
- **بستن برنامه های دیگر:** همان طور که گفته شد، اندروید اجازه Kill کردن برنامه های دیگر را نمی دهد؛ رفتار «بازگشت به صفحه اصلی و قفل صفحه» بهترین راه حل استاندارد است.

ساخت خروجی نهایی: از منوی Build گزینه **APK / Generate Signed App Bundle** را انتخاب کنید، یک keystore بسازید و APK نسخه release را تولید کنید. برای انتشار در Google Play حتماً در توضیحات اپ، دلیل نیاز به دسترسی ادمین (قفل خودکار صفحه) را شفاف بیان کنید؛ گوگل برای این دسترسی حساس فرآیند بررسی دارد و نبود توضیح شفاف باعث رد شدن اپ می‌شود.